

C 版 AIPL ユーザーズガイド

児玉靖司 (Yasushi Kodama)

法政大学 (Hosei University)

2026 年 5 月 12 日

目次

1	概要	2
2	対象実装	2
3	全体構成	2
4	abcl2c コマンド	2
5	POSIX pthread 版 C	3
5.1	主なランタイム構造	3
5.2	生成される処理	3
5.3	通常のビルド例	3
6	対応する AIPL 機能	4
7	未対応または制限のある機能	4
8	SDL2 GUI 付き C 版	4
8.1	代表的な GUI 組み込み	4
8.2	GUI ビルド例	5
9	Xinu 向け C 版	5
9.1	Xinu 版の特徴	5
9.2	Xinu 実行例	5
10	サンプル分類	6
11	テスト	6
12	まとめ	6

1 概要

C 版 AIPL は、AIPL の `.abcl` ソースを C コードへ変換して実行するためのコード生成系である。中心となるツールは `abcl2c` であり、実装は `src/abcl2c.ml` と `src/c_translator.ml` にある。

C 版は単一のランタイムではなく、用途別に以下の出力系を持つ。

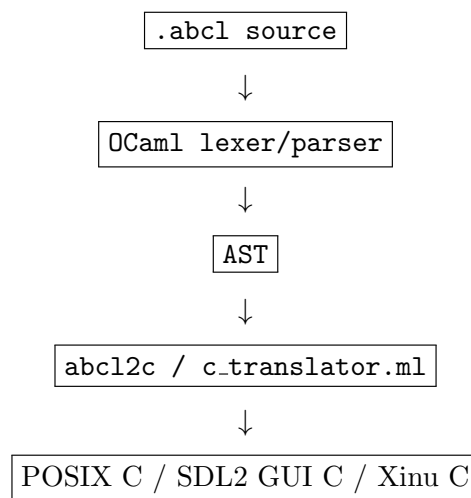
- POSIX pthread 版 C コード
- SDL2 GUI ランタイム付き C コード
- Xinu 向け C コード
- 参考実装としての Python コード生成

本ガイドでは、主に C 出力、SDL2 GUI 付き C 出力、Xinu 向け C 出力を扱う。

2 対象実装

項目	ファイル
変換器入口	<code>src/abcl2c.ml</code>
C 変換本体	<code>src/c_translator.ml</code>
GUI ランタイム	<code>src/abcl_gui_runtime.c</code>
OCaml パーサ	<code>src/parser.mly</code> , <code>src/lexer.mll</code>
AST 定義	<code>src/ast.ml</code>
GUI サンプル	<code>abclc/*Gui.abcl</code>
Xinu サンプル	<code>abclc/*Xinu.abcl</code>
実行スクリプト	<code>run_*-{gui,xinu}.sh</code>

3 全体構成



4 abcl2c コマンド

`abcl2c` は AIPL ソースを C または派生ターゲットへ変換する。

```
usage: abcl2c <input.abcl> [-o <output>] [--max-msgs N] [--xinu | --python]
```

オプション	意味
-o <output>	出力ファイル名を指定する。省略時は入力名から拡張子を変えて出力する。
--max-msgs N	実行時に処理するメッセージ数上限。0 で上限無効。
--xinu	Xinu 向け C コードを生成する。
--python	参考用の Python コードを生成する。

5 POSIX pthread 版 C

標準の C 出力は、POSIX pthread を使うアクターランタイムを内包する。各 AIPL アクターは C 側の `object_t` として管理され、各オブジェクトにメールボックスと pthread が割り当てられる。

5.1 主なランタイム構造

- `value_t`: `int`, `float`, `string`, `object id` を表す値。
- `message_t`: `sender`、`receiver`、`method`、引数列を持つメッセージ。
- `mailbox_t`: `mutex` と `condition variable` を持つリングバッファ型メールボックス。
- `object_t`: `class id`、`fields`、`mailbox`、`pthread` を持つアクター。
- `enqueue`: 受信側メールボックスへ非同期メッセージを投函する。
- `actor_main`: 各アクターのメッセージ処理ループ。

5.2 生成される処理

変換後の C コードでは、AIPL クラスごとに以下が生成される。

- `class id` の定義
- フィールド番号の `enum`
- フィールド初期化関数
- メソッド関数
- `method` 名による `dispatch` 関数
- グローバル `actor` 生成と起動処理

5.3 通常のビルド例

```
cd /Users/yaskodama/local-genai-chatgpt-ga/aios-claude
dune build src/abcl2c.exe
./_build/default/src/abcl2c.exe abclc/PingPong.abcl -o /tmp/pingpong.c --max-msgs 12
cc -O2 -Wall -pthread -o /tmp/pingpong /tmp/pingpong.c -lm
/tmp/pingpong
```

6 対応する AIPL 機能

C 版が主に扱う AIPL 機能は以下である。

機能	対応状況
<code>class / method</code>	C 関数と <code>dispatch</code> 関数へ変換される。
<code>var</code> フィールド	C 配列 <code>fields[]</code> の要素として保持される。
<code>new Class(args)</code>	C 側でオブジェクトを確保し、 <code>init</code> メッセージを投函する。
<code>send actor.method(args)</code>	<code>enqueue</code> に変換される。
<code>send!</code>	通常の <code>send</code> と同様に C 側投函へ変換される。
<code>self</code>	現在の <code>self_id</code> 。
<code>sender</code>	メッセージ送信元 ID。
<code>if / while</code>	C の制御構文へ変換される。
算術・比較	<code>v_binop</code> で処理される。
<code>print</code>	<code>v_print</code> で標準出力へ出す。
外部組み込み	C の <code>extern value_t name(int,value_t*)</code> としてリンクする。

7 未対応または制限のある機能

- `become` は C 出力ではコメント `/* become unsupported */` になる。
- `select` は C 出力ではコメント `/* select unsupported */` になる。
- `now / future / await` は C 出力の中心機能ではない。
- Python 版 AIPL の型注釈、所有権、線形型、`transient cast`、モデル検査機能は C 変換対象ではない。
- 標準ランタイムの固定上限として、`MAX_OBJECTS`, `MAX_MAILBOX`, `MAX_FIELDS`, `MAX_ARGS` がある。
- `non-object` なトップレベル変数は標準 C 出力では限定的な扱いになる。
- Xinu 版は POSIX 版よりさらに小さい固定上限を持つ。

8 SDL2 GUI 付き C 版

GUI 付き C 版は、通常の C 出力に `src/abcl_gui_runtime.c` をリンクして使う。SDL2 を使い、線描画、ボタン、スライダー、哲学者、bounded buffer、災害/ドローン風可視化などを提供する。

8.1 代表的な GUI 組み込み

関数	概要
<code>gui_open(w,h,title)</code>	GUI ウィンドウを開く準備をする。
<code>gui_set_line(...)</code>	線分を描画状態へ設定する。
<code>gui_add.button(...)</code>	GUI ボタンを追加する。
<code>gui_register_ticker(actor)</code>	定期的に動かす <code>actor</code> を登録する。

<code>gui_run()</code>	SDL イベントループを実行する。
<code>gui_dining_init(N)</code>	食事する哲学者の配置を初期化する。
<code>gui_set_phil(i,state)</code>	哲学者状態を更新する。
<code>gui_set_fork_held/free</code>	フォーク状態を更新する。
<code>gui_buf_setup(...)</code>	bounded buffer 表示を初期化する。
<code>gui_add_slider(...)</code>	スライダーを追加する。
<code>gui_slider_value(i)</code>	スライダー値を読む。

8.2 GUI ビルド例

```
cd /Users/yaskodama/local-genai-chatgpt-ga/aio-cs-2024
dune build src/abcl2c.exe
./_build/default/src/abcl2c.exe abclc/Rotate4LinesGui.abcl -o /tmp/r4l.c --max-msgs 0
cc -O2 -Wall -pthread $(pkg-config --cflags sdl2) \
  -o /tmp/r4l /tmp/r4l.c src/abcl_gui_runtime.c \
  $(pkg-config --libs sdl2) -lm
/tmp/r4l
```

既存スクリプトとして、以下が用意されている。

- `run_r4l_gui.sh`
- `run_p5_gui.sh`
- `run_bb_gui.sh`
- `run_disaster_gui.sh`

9 Xinu 向け C 版

`--xinu` を指定すると、Xinu 向け C コードを生成する。POSIX `pthread` の代わりに、Xinu の `thread`, `semaphore`, `kprintf`, `create`, `ready` を使うランタイムになる。

9.1 Xinu 版の特徴

- `pthread` ではなく Xinu `thread` を使う。
- `enqueue` は Xinu 内部名との衝突を避けて `abcl_enqueue` として実装される。
- エントリポイントは `thread aipl_main(void)` として生成される。
- QEMU 上の Xinu kernel へ組み込んで実行する想定である。

9.2 Xinu 実行例

```
cd /Users/yaskodama/local-genai-chatgpt-ga/aio-cs-2024
dune build src/abcl2c.exe
./_build/default/src/abcl2c.exe abclc/PingPong.abcl \
  -o /tmp/pp_xinu.c --xinu --max-msgs 12
```

生成した C ファイルを Xinu の `apps/` に配置し、Xinu kernel をビルドして QEMU で実行する。既存スクリプトとして以下がある。

- `run_pingpong_xinu.sh`
- `run_r4l_xinu.sh`
- `run_p5_xinu.sh`
- `run_bb_xinu.sh`

10 サンプル分類

サンプル	用途
<code>abclc/PingPong.abcl</code>	基本的なアクター間メッセージ。
<code>abclc/Rotate4LinesGui.abcl</code>	SDL2 GUI で回転線を描画。
<code>abclc/Philosophers5Gui.abcl</code>	GUI 付き食事する哲学者。
<code>abclc/BoundedBufferGui.abcl</code>	GUI 付き bounded buffer。
<code>abclc/DisasterReturnGui.abcl</code>	災害/帰還支援風 GUI サンプル。
<code>abclc/Rotate4LinesXinu.abcl</code>	Xinu 向け回転線サンプル。
<code>abclc/Philosophers5Xinu.abcl</code>	Xinu 向け哲学者サンプル。
<code>abclc/BoundedBufferXinu.abcl</code>	Xinu 向け bounded buffer。

11 テスト

`abclc/_smoke_test.sh` は、OCaml REPL で動かすサンプルと、`abcl2c` で変換すべき GUI/Python/Xinu 系サンプルを分けて検査する。GUI 系は `abcl2c` で C 生成後、`cc` と SDL2 でコンパイルする。

```
cd /Users/yaskodama/local-genai-chatgpt-ga/aio-cs-claude/abclc
./_smoke_test.sh
```

12 まとめ

C 版 AIPL は、AIPL プログラムをネイティブ C ランタイムへ落とし込むための実験的なコード生成系である。POSIX pthread 版では軽量なアクター実行を確認でき、SDL2 GUI ランタイムをリンクすれば可視化デモを構築できる。さらに Xinu 向け出力により、教育用 OS や組込み風環境で AIPL アクターを動かす実験も可能である。